

Software Testing Overview

What is software testing?

- Software testing is an investigation conducted to provide stakeholders with information about the quality of the product or service under test



White box vs. Black box testing

Black-box testing

- Aka specification testing
- Test case generation are based solely on the knowledge of the system requirements.
- Ensures that specified features of the software are addressed by some aspect of the program



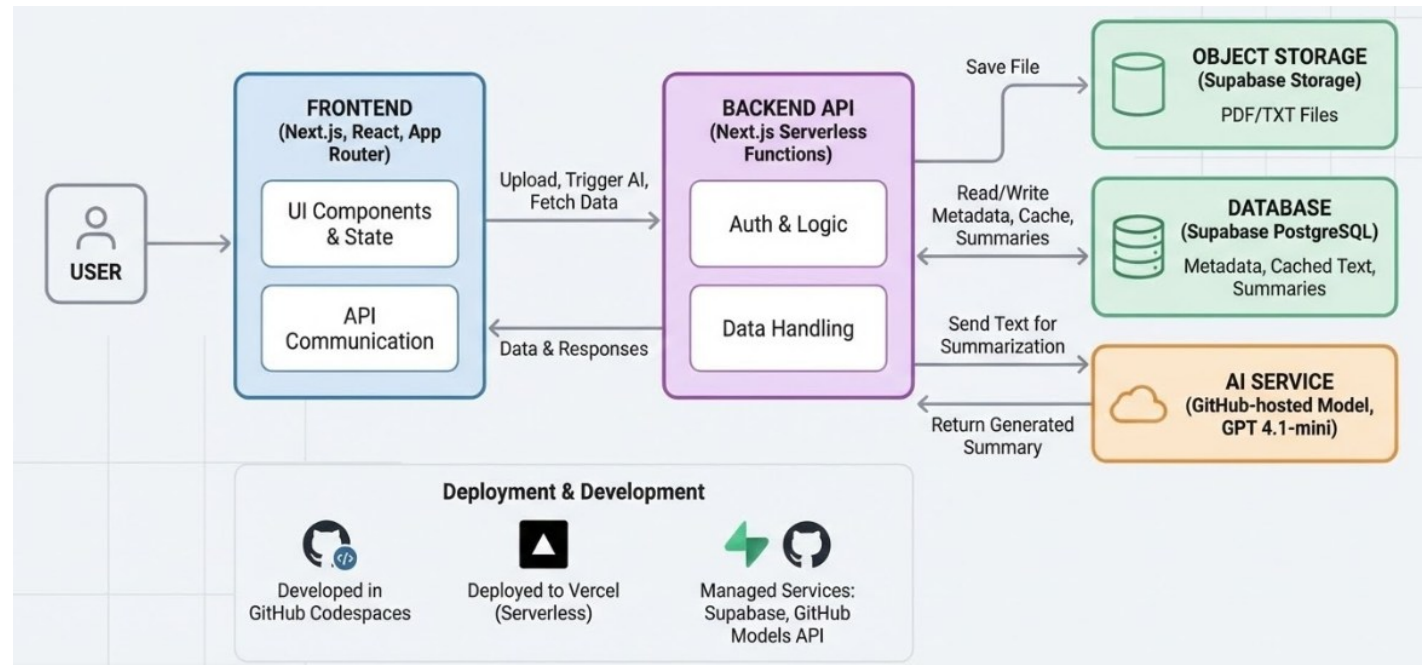
White-box testing

- Aka Code-based testing
- Test data selection is guided by information derived from the internal structure and internal functional properties of the program

```
if (testscore >= 90) {  
    grade = 'A';  
} else if (testscore >= 80) {  
    grade = 'B';  
} else if (testscore >= 70) {  
    grade = 'C';  
} else if (testscore >= 60) {  
    grade = 'D';  
} else {  
    grade = 'F';  
}
```

Happy Path Testing

- Happy path testing verifies that software works correctly when users follow the intended workflow with valid inputs and expected behavior
- User uploads a supported .pdf file within size limits; app parses and sends it to the LLM successfully



Edge case testing

- Focuses on *extreme* but *often valid inputs* at the boundaries of normal behavior
- Examples
 - Upload a PDF that is exactly at the max file size limit of Supabase
 - Upload a file with a very long filename
 - Send a document that hits the LLM's max context window limit
 - User rapidly clicks "Summarize" multiple times before the first response returns
 - User navigates away mid-upload, then returns to the same page

Global file size

You can set the maximum file size across all your buckets by setting the *Global file size limit* value in your **Storage Settings**. For Free projects, the limit can't exceed 50 MB. On the Pro Plan and up, you can set this value to up to 500 GB. If you need more than 500 GB, [contact us](#).

Plan	Max File Size Limit
Free	50 MB
Pro	500 GB
Team	500 GB
Enterprise	Custom

<https://supabase.com/docs/guides/storage/uploads/file-limits>

Rate limit tier	Rate limits	Copilot Free	Copilot Pro	Copilot Business	Copilot Enterprise
Low	Requests per minute	15	15	15	20
	Requests per day	150	150	300	450
	Tokens per request	8000 in, 4000 out	8000 in, 4000 out	8000 in, 4000 out	8000 in, 8000 out
	Concurrent requests	5	5	5	8
High	Requests per minute	10	10	10	15
	Requests per day	50	50	100	150
	Tokens per request	8000 in, 4000 out	8000 in, 4000 out	8000 in, 4000 out	16000 in, 8000 out
	Concurrent requests	2	2	2	4

<https://docs.github.com/en/github-models/use-github-models/prototyping-with-ai-models#rate-limits>

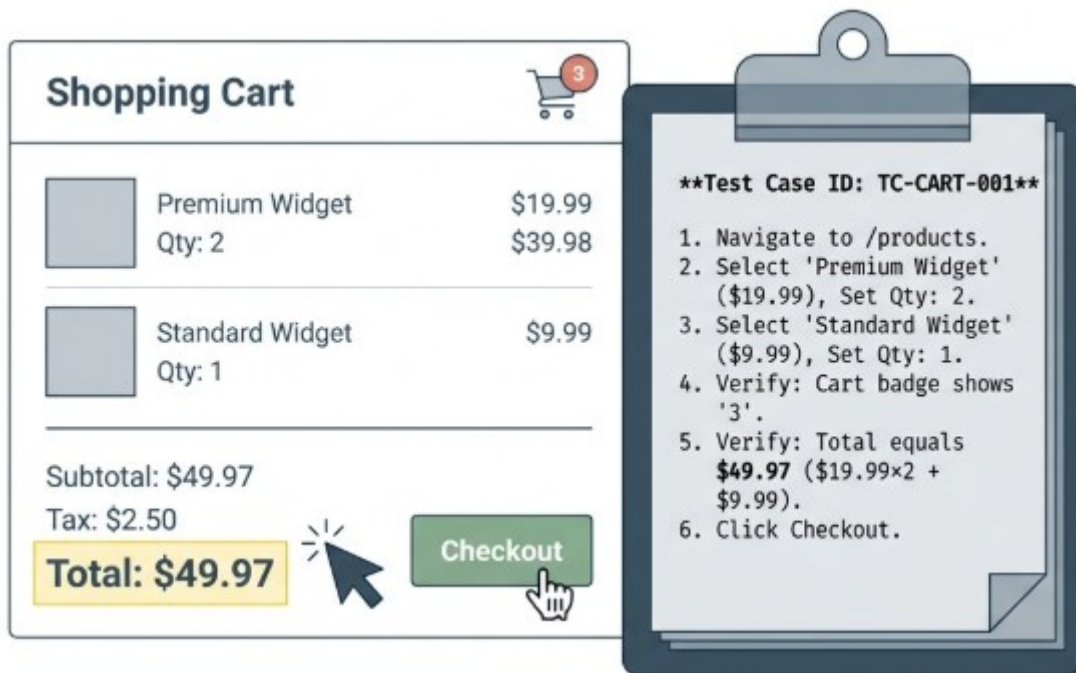
Negative testing

- Focuses on invalid inputs and error conditions (anything the system should reject or handle gracefully)
- Examples
 - Upload an unsupported file type (e.g., .exe, .mp4, .zip)
 - Upload a corrupted/broken PDF that cannot be parsed
 - Upload a file exceeding the size limit
 - Exceptions when calling LLM APIs

HTTP Status Code	Error Type / Reason	Description
200 OK	Normal Success	Your request worked fine, and you got the response you asked for.
400 Bad Request	Content Filter	Request blocked due to prompt or completion violating content policies (hate, violence, sexual, etc.).
400 Bad Request	Invalid Request	Something was wrong with your request, like missing or incorrect information.
429 Too Many Requests	Rate Limit	You are sending too many requests or too much data too quickly
500 Internal Server	Server Error	Something went wrong on the server side, not because of your request. Try again later.

Manual testing

- A human tester executes test cases by hand without automation tools.



Test Steps:

- Open browser and navigate to <http://localhost:8080/products>
- Manually select a product priced at \$19.99
- Change quantity to 2 using the quantity selector
- Click "Add to Cart" button
- Select another product priced at \$9.99
- Keep quantity as 1
- Click "Add to Cart" button
- Click "View Cart" link
- Observe the cart page

Verification:

- Visually check that cart displays "3 items"
- Visually verify that total amount shows "\$49.97"
($2 \times \$19.99 + 1 \times \$9.99 = \$49.97$)
- Check that product names and images display correctly
- Verify buttons are clickable and responsive

Strength/Weakness of manual testing

- Strength
 - Usability testing: Evaluating if the interface is intuitive and easy to use
 - Visual validation: Checking layouts, colors, fonts, and responsive design
 - Exploratory testing: Discovering unexpected bugs by trying edge cases
 - User experience: Assessing the overall feel and flow of the application
- Weakness
 - Time-consuming
 - Error-prone (humans make calculation mistakes)
 - Not repeatable (different testers may execute differently)

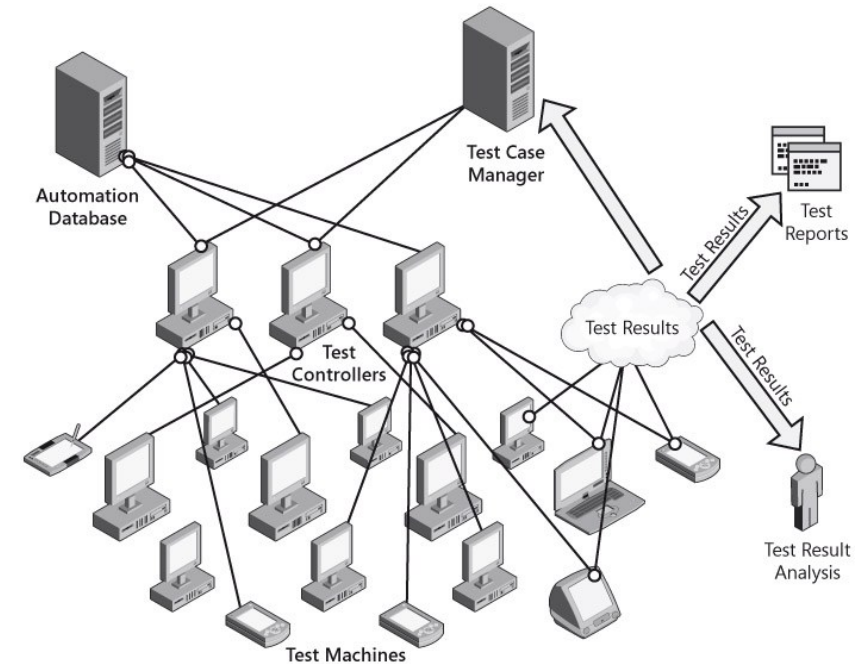
Automated testing

- The use of software to control the execution of tests
- Advantages
 - Faster and parallel execution
 - Eliminates human error by executing the exact same steps consistently every run
 - Write once, run many times across multiple test cycles and projects
- Best for regression, load, CI/CD pipeline testing

```
test_cart.py > ...
1 def test_addToShoppingCart():
2     # Test case 1: Adding an item to an empty cart
3     carts = []
4     item = "apple"
5     expected_output = ["apple"]
6     assert addToShoppingCart(item) == expected_output
7
8     # Test case 2: Adding an item to a non-empty cart
9     carts = ["apple", "banana"]
10    item = "orange"
11    expected_output = ["apple", "banana", "orange"]
12    assert addToShoppingCart(item) == expected_output
13
14    # Test case 3: Adding an item to a cart with duplicate items
15    carts = ["apple", "banana", "apple"]
16    item = "apple"
17    expected_output = ["apple", "banana", "apple", "apple"]
18    assert addToShoppingCart(item) == expected_output
19
20    # Test case 4: Adding an item to a cart with different types of items
21    carts = ["apple", 123, True]
22    item = "banana"
23    expected_output = ["apple", 123, True, "banana"]
24    assert addToShoppingCart(item) == expected_output
25
```

Advantages of automated testing

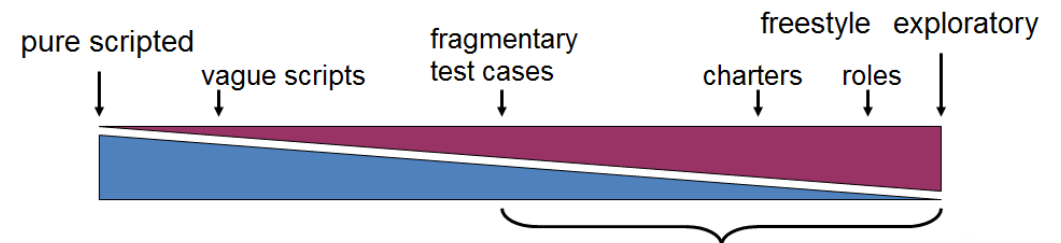
- Fast
 - Automated Tools run tests significantly faster than human users
 - Good for load testing, massive random testing, etc
- Reliable
 - Tests perform precisely the same operations each time they are run, thereby eliminating human error
- Repeatable
 - You can test how the software reacts under repeated execution of the same operations.



Exploratory Testing

- An approach to software testing that emphasizes simultaneous learning, test design, and test execution.
- Allows testers to investigate the application dynamically, adapting their testing strategy based on what they discover during the testing process

The Scripted/Exploratory Testing Continuum



When I say “exploratory testing” and don’t qualify it, I mean anything on the exploratory side of this continuum.

<https://www.satisfice.com/blog/archives/1509>

Session Based Exploratory Testing (SBET)

- Learning Phase
 - Understand the application, its intended functionality, and potential risk areas.
 - Gather information from available documentation, user stories, etc., to build context for their exploration
- Test Charter Creation
 - A test charter serves as a mission statement for the exploratory testing session
 - The charter defines what will be tested, how it will be tested, and what potential problems to look for
- Session Execution
 - During this phase, testers actively explore the application, simultaneously designing and executing tests based on their observations.
 - Document findings, investigate anomalies, and adapt the testing approach based on discoveries.
- Analysis and Reporting
 - After completing the testing session, testers analyze their findings, document issues, and prepare reports for stakeholders.
 - Create bug reports, session summaries, and recommendations for future testing.

Timeboxing

- Set a duration
 - Decide on a specific time limit for the session, typically ranging from 60 to 90 minutes.
 - Ensure that testers can concentrate without distractions during the timebox.
- Work in pairs: Have two testers work together during the session.
 - This collaborative approach can enhance creativity and lead to more thorough exploration.
- Be flexible with the timebox duration based on test progression.
 - If you find critical issues, you might extend the session by up to 45 minutes, or shorten it if you've covered everything needed.

Example of an exploratory test session

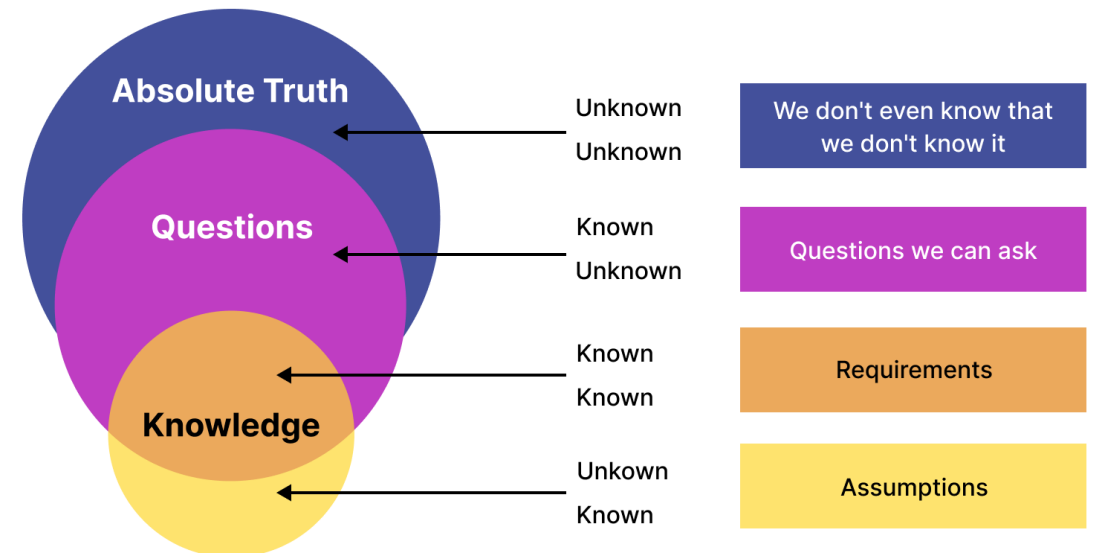
Scenario: You're a tester for a web-based e-commerce application.

- Test charter
 - Your goal is to test the checkout process for usability and functionality.
- Exploration
 - Begin by familiarizing yourself with the application's main features, such as product browsing, cart management, and payment processing.
- Execution
 - Add items to your shopping cart and proceed to checkout. Pay close attention to any issues, such as the cart not updating correctly when items are added or removed.
 - Document any bugs you encounter, including:
 - Include screenshots of the issues
 - Descriptions of the unexpected behavior.
 - Adapt your testing based on discoveries.
 - E.g., if you notice inconsistencies in how items are managed in the cart, explore that functionality further by trying different combinations of adding and removing items.

Why exploratory testing?

- Scripted Testing can only catch bugs that we know may happen (we can only create test scripts for something we know about)
- Exploratory testing catches bugs that we don't even know exist in the first place (Unknown Unknown)

Category	What It Means	Example in E-commerce App Testing
Known Known	You know it, and you know you know it.	The checkout page should display a confirmation after successful payment. You have tested this before and expect it to work.
Known Unknown	You know there's something you don't know, and you're aware of it.	You're unsure how the app handles coupon codes for expired promotions, so you plan to explore this scenario.
Unknown Known	Something is known (by someone or in documentation), but you're not aware of it.	There's a documented workaround for a rare payment gateway error, but you haven't read about it, so you might miss testing it.
Unknown Unknown	You don't realize what you don't know—hidden risks or behaviors.	The app crashes when a user enters emojis in their shipping address—a bug no one anticipated or scripted for.

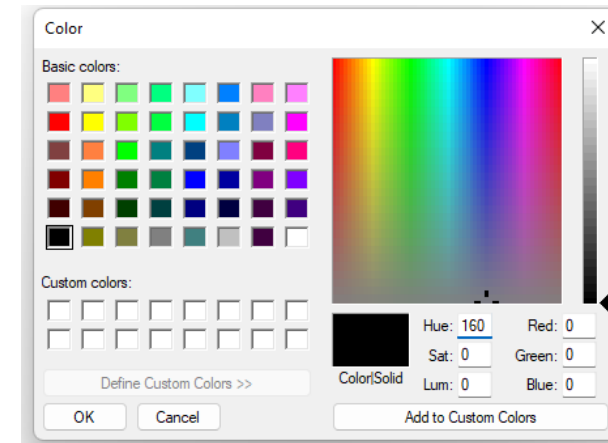
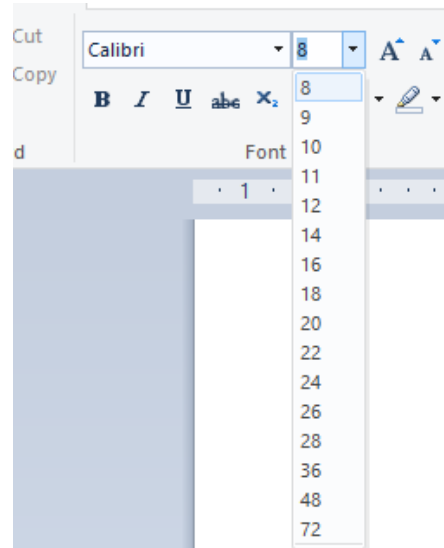
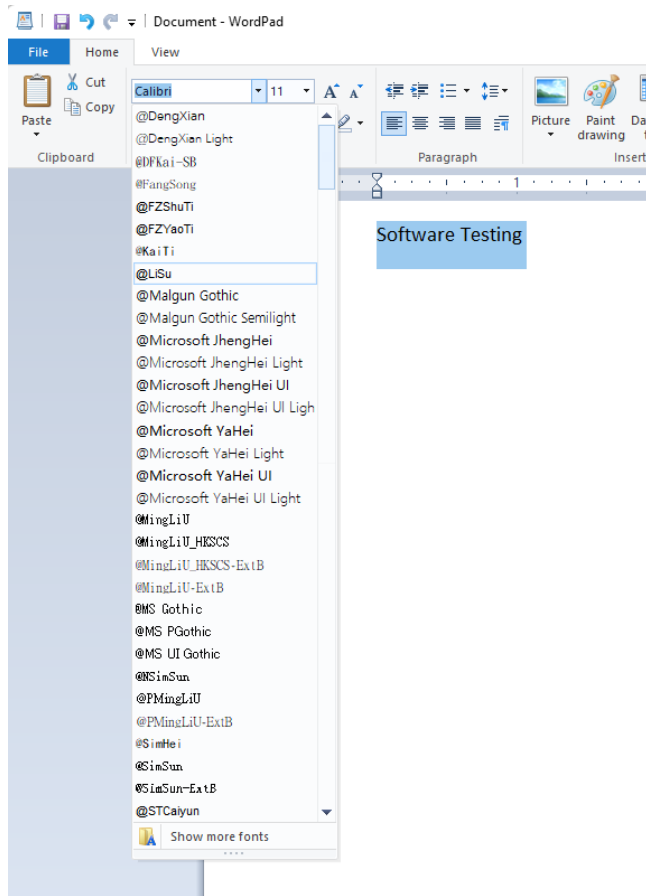


<https://katalon.com/resources-center/blog/exploratory-testing>

Scripted vs. Exploratory Testing

Aspect	Scripted Testing	Exploratory Testing
Test Case Creation	Predefined test cases	No predefined test cases
Test Execution	Follows scripted steps	Ad-hoc, unscripted
Test Planning	Detailed test plans required	Less formal test planning
Flexibility	Less flexible	Highly flexible
Test Documentation	Extensive documentation	Minimal documentation
Test Coverage	Specific and limited	Wider test coverage
Automated Testing	Highly suitable for automated testing	Not suitable for automation (requires human instinct and curiosity)
Bug Discovery	Less effective in finding new issues	Effective in uncovering hidden issues
Use Cases	Suitable for repetitive tasks	Suitable for new feature testing

What are the test cases in WordPad?



Complete testing is impossible

- Testing everything (all combinations of input and preconditions) is not feasible except in trivial cases.
- The number of possible inputs is very large.
- The number of possible outputs is very large.
- The number of paths through the software is very large.
- The software specification is subjective. You might say that a bug is in the eye of the beholder.

Instead of running all the test cases, we select only the most representative test cases to test the program!

What is a successful software testing?

- If no defect is found in testing, can we conclude that the software is defect-free?
- If the testing does not reveal any bug, may be it's just the case that the test case is not good enough to reveal the bug!
- Software testing can only show the presence of defects
- A “successful” test is one that can detect as many defects as possible
 - Prerequisite for locating and eliminating the defects, which may lead to an improvement in software quality
- More testing => more confidence that software is of good quality

*Software testing should **NOT** be used
to show that the software work.*

Risk based testing

- Prioritizes testing efforts based on the probability and impact of potential failures
- Focusing the “What breaks most often, and what's the cost if it does?”
 - Higher risk score → more testing effort;
 - Lower risk score → lighter or deferred testing

Risk Priority Number = Probability × Impact

Example

#	Feature	Probability	Impact	RPN	Risk Level
1	Payment Processing	2 / 5	5 / 5	10 / 25	● Critical
2	Authentication & Authorization	2 / 5	5 / 5	10 / 25	● Critical
3	Order Placement & Checkout	3 / 5	5 / 5	15 / 25	● Critical
4	Inventory & Stock Management	3 / 5	4 / 5	12 / 25	● Critical
5	Shopping Cart	3 / 5	4 / 5	12 / 25	● Critical
6	Product Search & Filtering	3 / 5	3 / 5	9 / 25	● High
7	Promo Codes & Discounts	3 / 5	3 / 5	9 / 25	● High
8	Order History & Tracking	2 / 5	3 / 5	6 / 25	● Medium
9	User Profile & Address Management	2 / 5	3 / 5	6 / 25	● Medium
10	Product Reviews & Ratings	2 / 5	2 / 5	4 / 25	● Low
11	Wishlist	2 / 5	1 / 5	2 / 25	● Low

Probability Scale (1–5)

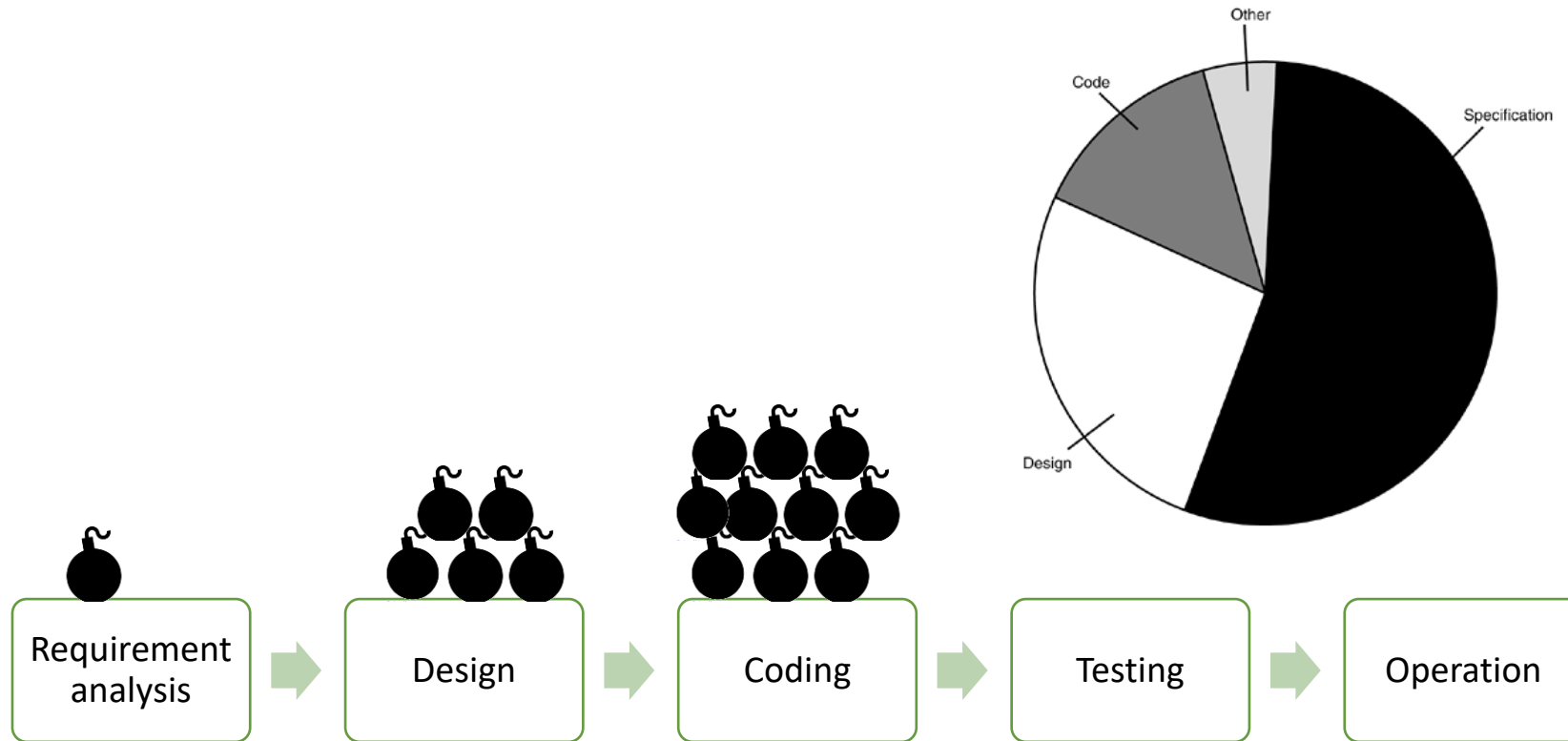
Score	Level	Description	Software Testing Indicators
5	Almost Certain	Failure happens regularly	Unstable code, no tests, frequent changes, complex business rules
4	Likely	Failure happens often	Many dependencies, high-traffic feature, known past bugs
3	Possible	Failure happens occasionally	Moderate complexity, some test coverage, occasional issues
2	Unlikely	Failure is rare	Stable, well-tested, simple logic, few dependencies
1	Rare	Failure almost never happens	Simple CRUD, isolated feature, heavily tested, rarely changed

Impact Scale (1–5)

Level	RPN Range	Action
● Critical	10–25	Test first, test deeply, automate
● High	7–9	Test thoroughly, include in regression
● Medium	5–6	Test in standard cycles
● Low	1–4	Test last or defer

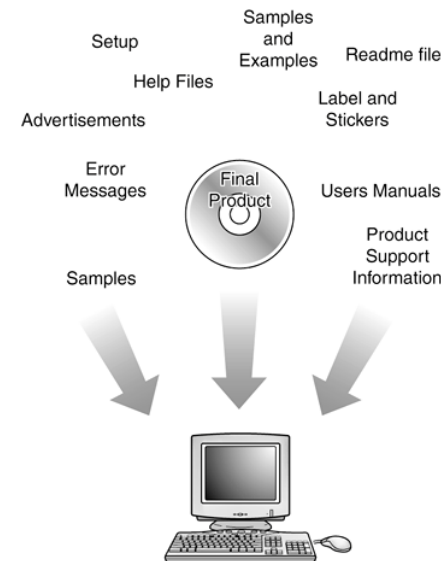
Sources of software defect

- Numerous studies have been performed to show that the main cause of software defect is specification and design

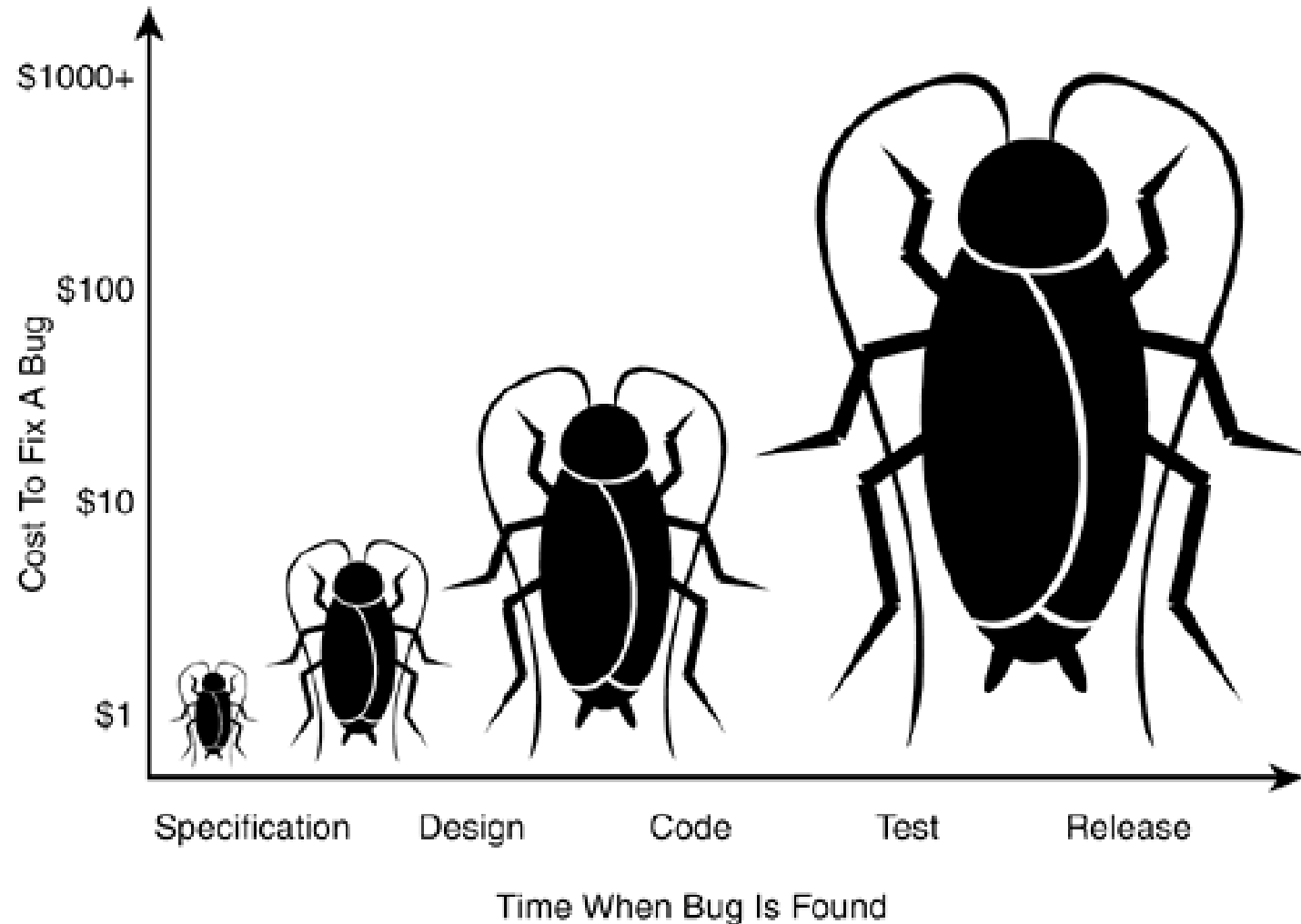


Dynamic testing vs. static testing

- Dynamic testing
 - Detect the presence of defects by running the program with a given set of test cases
- Static testing
 - Non-software pieces may also have bugs and should be tested
 - Refers to testing something that's not running
 - E.g. Review the code and documentations (e.g. requirement specification, design, user manual, test plan and test cases) to locate errors

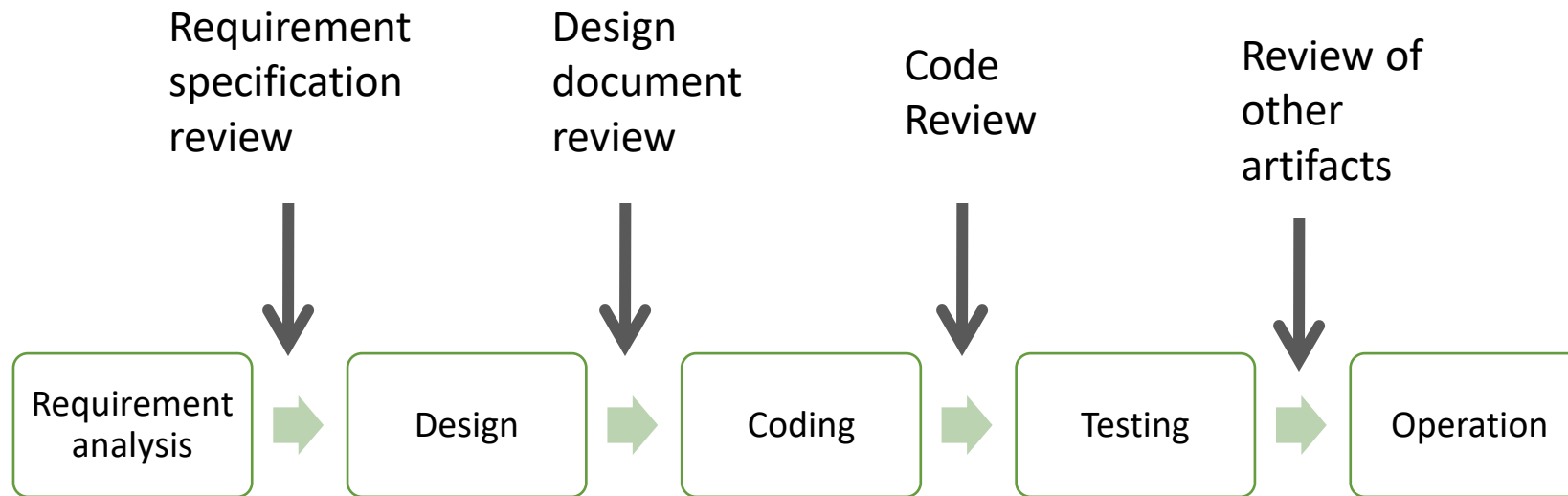


The cost to fix bugs



Static testing

- Static testing refers to testing something that's not running
 - Examples of activities
 - Review the code and documentations (e.g. requirement specification, design, user manual, test plan and test cases) to locate errors



Verifiable/Testable requirements

- A requirement is verifiable if, and only if, there exists some finite cost-effective process with which a person or machine can check that the software product meets the requirement.
- Non-verifiable requirements include statements such as “works well,” “user friendly,” and “shall usually happen.”

Exercise: Review the requirements using the defect checklist

Completeness

- ☐ Do the requirements address all known customer or system needs?
- ☐ Is any needed information missing? If so, is it identified as TBD?
- ☐ Have algorithms intrinsic to the functional requirements been defined?
- ☐ Are all external hardware, software, and communication interfaces defined?
- ☐ Is the expected behavior documented for all anticipated error conditions?
- ☐ Do the requirements provide an adequate basis for design and test?
- ☐ Is the implementation priority of each requirement included?
- ☐ Is each requirement in scope for the project, release, or iteration?

Correctness

- ☐ Do any requirements conflict with or duplicate other requirements?
- ☐ Is each requirement written in clear, concise, unambiguous, grammatically correct language?
- ☐ Is each requirement verifiable by testing, demonstration, review, or analysis?
- ☐ Are any specified error messages clear and meaningful?
- ☐ Are all requirements actually requirements, not solutions or constraints?
- ☐ Are the requirements technically feasible and implementable within known constraints?

Quality Attributes

- ☐ Are all usability, performance, security, and safety objectives properly specified?
- ☐ Are other quality attributes documented and quantified, with the acceptable trade-offs specified?
- ☐ Are the time-critical functions identified and timing criteria specified for them?
- ☐ Have internationalization and localization issues been adequately addressed?
- ☐ Are all of the quality requirements measurable?

Organization and Traceability

- ☐ Are the requirements organized in a logical and accessible way?
- ☐ Are all cross-references to other requirements and documents correct?
- ☐ Are all requirements written at a consistent and appropriate level of detail?
- ☐ Is each requirement uniquely and correctly labeled?
- ☐ Is each functional requirement traced back to its origin (e.g., system requirement, business rule)?

Other Issues

- ☐ Are any use cases or process flows missing?
- ☐ Are any alternative flows, exceptions, or other information missing from use cases?
- ☐ Are all of the business rules identified?
- ☐ Are there any missing visual models that would provide clarity or completeness?
- ☐ Are all necessary report specifications present and complete?

The screenshot shows a flight booking application interface. At the top, there is a progress bar with four steps: 1 Search, 2 Select, 3 Details, and 4 Payment. The main heading is 'Book Your Flight'. Below this, there are radio buttons for 'Round-trip' (selected), 'One-way', and 'Multi-city'. The form includes fields for 'From' (Hong Kong (HKG)), 'To' (Los Angeles (LAX)), 'Departure Date' (10/06/2025), and 'Return Date' (10/12/2025). There are also fields for 'Passengers' (1 Passenger), 'Cabin Class' (Economy), and 'Currency' (HKD). A large green button labeled 'Search Flights' is at the bottom of the form.

Functional Requirements:

<https://gist.github.com/cswclui/9a1e10149580ebadb2ce37fecee40648>

The App

https://polyurichard.github.io/lectures/example/airticket_booking.html

Exercise: What test cases should be used for testing the app?

Example: Testable non-functional requirement

The system should be easy to use by medical staff and should be organized in such a way that user errors are minimized

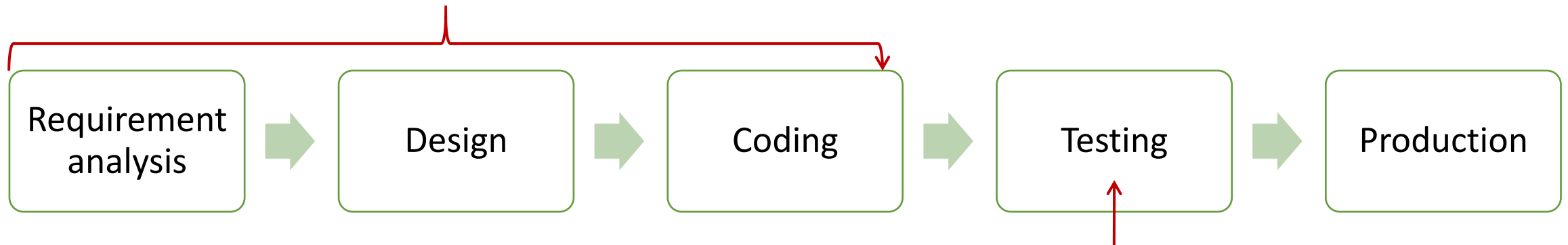


Medical staff shall be able to use all the system functions after four hours of training. After this training, the average number of errors made by experienced users shall not exceed two per hour of system use.

Defect Removal Efficiency (DRE)

- A good testing process should
 - Increase the number of defects found by tester and decrease the number of defects found by customers
 - Detect the defect as early as possible
- DRE measures the effectiveness of a test process or a testing phase

These activities can inject defects, while the reviews and inspections at the end of the phase can remove defects.



The testing phase is mainly for defect removal. There is also a chance to introduce defects when the found defects are fixed incorrectly!

Effectiveness of a test process

$$DRE = \frac{\textit{no. of defects detected (and removed)}}{\textit{total no. of defects}}$$

- We normally use the first year after product release to quantify the total defects.
- For example, suppose 1000 defects were detected during the development, and 100 defects were reported within the first year of the system release, then the total defect count is 1100.

Exercise

System	Defects found in testing	Number of defects found by users
A	30	70
B	20	30

- Which test team is more effective in finding defects?